Banker's Deadlock Avoidance Algorithm

Completed

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

The first line of input contains an integer representing the number of processes.
The second line contains an integer representing the number of resource types.
The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
This is followed by the Maximum Demand (Max) matrix with the same dimensions.
The final line contains the Available resources vector, where each value represents the number of currently available instances of

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    int n, m;

    printf("Enter number of processes: \n");
    if (scanf("%d", &n) != 1) return 0;
    printf("Enter number of resource types: \n");
    if (scanf("%d", &m) != 1) return 0;

    int alloc[n][m], max[n][m], avail[m], need[n][m];

    printf("Enter Allocation Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
```

Test Cases 2

Run Sample Cases

Run All

Custom Test Case

> Custom Test

Banker's Deadlock Avoidance Algorithm

Completed

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the number of resource types.
- The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
- This is followed by the Maximum Demand (Max) matrix with the same dimensions.
- The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c

    printf("Enter Max Matrix:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter Available Resources: \n");
    for (int j = 0; j < m; j++) {
        scanf("%d", &avail[j]);
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
```

Test Cases 2

Run Sample Cases

Run All Cas

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Banker's Deadlock Avoidance Algorithm

Completed

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate hat a deadlock may occur.

put Format

e first line of input contains an integer representing the number processes.
e second line contains an integer representing the number of ource types.
e next lines contain the Allocation matrix, where each row responds to a process and each column corresponds to a ource type.
is followed by the Maximum Demand (Max) matrix with the e dimensions.
final line contains the Available resources vector, where each e represents the number of currently available instances of resource type.

Select Language : C (GCC 9.2.0)

```c
    bool finish[n];
    for (int i = 0; i < n; i++) finish[i] = false;

    int safeSeq[n];
    int count = 0;

    while (count < n) {
        bool found = false;
        for (int i = 0; i < n; i++) {
            if (!finish[i]) {
                bool possible = true;

                for (int j = 0; j < m; j++) {
                    if (need[i][j] > avail[j]) {
                        possible = false;
                        break;
                    }
                }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Banker's Deadlock Avoidance Algorithm

Completed

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

1. The first line of input contains an integer representing the number of processes.
2. The second line contains an integer representing the number of resource types.
3. The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
4. This is followed by the Maximum Demand (Max) matrix with the same dimensions.
5. The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c
                if (possible) {

                    for (int j = 0; j < m; j++) {
                        avail[j] += alloc[i][j];
                    }
                    safeSeq[count++] = i;
                    finish[i] = true;
                    found = true;
                }
            }
        }

        if (!found) break;
    }

    if (count == n) {
        printf("System is in a SAFE STATE.\n");
        printf("Safe Sequence: ");
```

Test Cases 2

Run Sample Cases

Run All Cas

Custom Test Case

> Custom Test

Sample Test Case

> Test Case - 1

Completed

Banker's Deadlock Avoidance Algorithm

Write a C program to implement the Banker's Deadlock Avoidance Algorithm. The program should accept the number of processes and resource types, the Maximum Demand matrix, the currently Allocated resources matrix, and the Available resources vector. Calculate the 'Need' matrix and determine if the system is in a 'Safe State'. If it is safe, display the Safe Sequence; otherwise, indicate that a deadlock may occur.

Input Format

The first line of input contains an integer representing the number of processes.
The second line contains an integer representing the number of resource types.
The next lines contain the Allocation matrix, where each row corresponds to a process and each column corresponds to a resource type.
This is followed by the Maximum Demand (Max) matrix with the same dimensions.
The final line contains the Available resources vector, where each value represents the number of currently available instances of each resource type.

Select Language : C (GCC 9.2.0)

```c
        }

        if (!found) break;
    }

    if (count == n) {
        printf("System is in a SAFE STATE.\n");
        printf("Safe Sequence: ");
        for (int i = 0; i < n; i++) {
            printf("P%d ", safeSeq[i]);
        }
        printf("\n");
    } else {
        printf("System is NOT in a safe state (Deadlock may occur).\n");
    }

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case